Operating System (OS): Process ID (PID)

1. What is Process ID (PID)?

Definition:

A **Process ID (PID)** is a **unique identification number** assigned by the Operating System to every running process. It allows the OS to uniquely identify, manage, and control each process.

Example

Process Name	Process ID (PID)
`chrome.exe`	**2548**
`notepad.exe`	**7352**
`explorer.exe`	**2456**

At any given time, no two running processes have the same PID. Once a process is terminated, its PID may be reused by the operating system for a new process.

Why is PID Important?

- **Process Identification:** Identifies each running process uniquely.
- **Process Management:** Allows the OS to monitor and manage processes.
- **Process Termination:** Enables users or the OS to terminate a specific process using its PID.
- **Parent–Child Relationship:** Maintains relationships between parent and child processes.

---

2. What is Process Suspension?

Definition:

Process Suspension is the process of **temporarily stopping (pausing) the execution of a process without terminating it**. The suspended process can later continue execution from the same point.

Process State Transition

A process does not become suspended instantly. It goes through the following transition:

Running
↓
Suspending
↓
Suspended

Suspend vs. Terminate

Suspend	Terminate
Temporarily stops the process.	Permanently ends the process.
The process can be resumed later.	The process cannot be resumed.
The process state is preserved.	The process state is destroyed.
CPU usage becomes 0% while suspended.	All allocated resources are released.

---

3. What Happens Internally When a Process is Suspended?

When the operating system suspends a process, three major operations take place.

A. State Change

The process state changes to either:

- **Suspended Ready**
- **Suspended Blocked**

B. Resource and CPU Release

The operating system removes the process from CPU execution.

- CPU usage becomes **0%**.
- CPU time becomes available for other processes.

C. Context Saving

The operating system saves the complete execution context of the process, including:

- CPU Registers
- Program Counter (PC)
- Stack Pointer
- Stack
- Other processor state information

This allows the process to continue later from exactly where it stopped.

---

4. Where is the Process State Stored?

All important information about a process is stored in a special operating system data structure called the **Process Control Block (PCB).**

RAM (Main Memory)
│
└── Kernel Space
    │
    └── Process Control Block (PCB)
        ├── PID
        ├── Process State
        ├── CPU Registers
        ├── Program Counter
        ├── Scheduling Information
        └── Memory Management Information

Memory Location Details

PCB Location

The **PCB is stored in the Kernel Space of RAM**.

Even if a process is suspended, its PCB normally remains in RAM so that the operating system can manage and resume the process quickly.

Memory Swapping (Swap Out)

If the system experiences **low memory**, the operating system may move the process's memory image (such as code, data, stack, and heap) from RAM to secondary storage.

- **Linux:** Swap Space
- **Windows:** Pagefile

This operation is called **Swap Out**.

Important Note:

- **PCB remains in RAM.**
- **Only the process memory (code, data, stack, heap) may be moved to disk when necessary.**

---

5. Why Does the Operating System Suspend Processes?

The operating system suspends processes for several reasons.

Memory Management

When RAM becomes insufficient, less important background processes may be suspended to free memory.

Power Saving

Suspended processes consume almost no CPU time, helping reduce power consumption and extend battery life.

Example:

Background applications on Windows and smartphones are often suspended automatically.

Deadlock Recovery

If a deadlock or resource conflict occurs, the operating system may suspend one or more processes temporarily to recover system resources and maintain system stability.

---

6. Practical Commands and Usage

Linux / Unix

Suspend a Process

Press:

Ctrl + Z

This sends the **SIGSTOP** signal to suspend the running process.

Resume a Process

Use:

fg

This sends the **SIGCONT** signal and resumes the suspended process in the foreground.

Windows

Task Manager

You can suspend or resume certain processes through:

Task Manager
↓
Performance
↓
Open Resource Monitor
↓
Right-click Process

↓
Suspend Process / Resume Process

Sleep Mode

Sleep Mode is a practical example of process suspension.

When the computer enters Sleep Mode:

- Running processes are suspended.
- Their execution state remains stored in RAM.
- The computer enters a low-power state.
- When the computer wakes up, the processes resume from where they were suspended.

Exam Definition (Short)

Process ID (PID)

> **A Process ID (PID) is a unique identification number assigned by the operating system to every running process for identification and management.**

Process Suspension

> **Process Suspension is the temporary stopping of a process without terminating it, while preserving its execution state so that it can be resumed later.**

Yesterday 11:11 PM

ভালো প্রশ্ন। এগুলো দুইটি state-এর মধ্যে ভাগ করে বুঝতে হবে।

Embryo (New/Starting) State-এ যা হয়

- ✅ The process has already been created.
- ✅ A PID has been assigned.
- ✅ The PCB (Process Control Block) has been created.
- ✅ Memory has been allocated to the process.

- ✅ Required resources (memory, files, etc.) are allocated.

এরপর Process Ready Queue-তে পাঠানো হয়।

Ready State-এ যা হয়

- ✅ The process is placed in the Ready Queue.
- ✅ The process is waiting for the CPU Scheduler to assign the CPU.
- ❌ CPU ব্যবহার করছে না।
- ❌ কোনো instruction execute হচ্ছে না।
- ✅ CPU পেলেই Running State-এ চলে যাবে।

সহজে মনে রাখো

Embryo/New State = Process তৈরি করার কাজ

- PID তৈরি
- PCB তৈরি
- Memory Allocate
- Resource Allocate

Ready State = Process প্রস্তুত, শুধু CPU-এর অপেক্ষা করছে

- Ready Queue-তে থাকে
- Scheduler-এর জন্য অপেক্ষা করে
- CPU পেলেই Running হবে

এটাই পরীক্ষার জন্য সবচেয়ে সঠিকভাবে দুই state-এর পার্থক্য।

Running state bolo

Running State

Definition

Running State is the state in which a process is currently executing instructions using the CPU.

What happens in the Running State?

- ✅ The process is assigned the CPU by the CPU Scheduler.
- ✅ The CPU executes the process instructions.
- ✅ The Program Counter (PC) continuously updates to point to the next instruction.

- ✅ CPU register values are continuously updated.
- ✅ The process may access memory, files, and I/O devices.
- ✅ The process performs computations and executes its assigned tasks.

CPU Status

- ✅ The process is using the CPU.
- ✅ Instructions are being executed.
- ❌ The process is not waiting for the CPU.

Possible Transitions from Running State

1. Running → Ready

- The process's time quantum (time slice) expires.
- The CPU Scheduler removes it from the CPU (descheduling/preemption).

2. Running → Blocked

- The process requests an I/O operation (e.g., keyboard input, disk read, network communication).
- It waits until the I/O operation is completed.

3. Running → Ending (Zombie)

- The process completes its execution.
- It enters the Ending/Zombie state before being removed from the system.

Simple Flow

```
Ready
   │
Scheduler assigns CPU
   ▼
Running
   ├──→ Ready (Time slice expires)
   ├──→ Blocked (Waiting for I/O)
   └──→ Ending/Zombie (Execution completed)
```

Exam Definition

Running State is the state in which a process is actively executing instructions using the CPU.

zombie state...

Zombie State (Ending State)

Definition

Zombie State is the state in which a process has finished its execution, but its Process Control Block (PCB) and exit status are still kept by the operating system until the parent process collects them.

What happens in the Zombie State?

- ✅ The process has completed its execution.
- ✅ The process does not use the CPU.
- ✅ The process does not execute any instructions.
- ✅ Most of the process memory and other resources are released.
- ✅ Only the PCB and exit status remain in the kernel.
- ✅ The operating system waits for the parent process to collect the child's exit status using `wait()` or `waitpid()`.

CPU Status

- ❌ CPU is not used.
- ❌ No instructions are executed.

Memory Status

- ✅ Most memory (code, data, stack, heap) is released.
- ✅ Only the PCB and exit information remain in kernel memory (RAM).

Why does a Zombie Process exist?

A zombie process exists because the parent process has not yet collected the child's termination status. After the parent process collects this information, the operating system completely removes the process and deletes its PCB.

Transition

`Running`

```
    |
    ▼
Zombie (Ending)
    |
Parent collects exit status
    ▼
Destroyed (Removed from the system)
```

Exam Definition

Zombie State is the state in which a process has completed execution, but its PCB and exit status remain in the operating system until the parent process collects them.

Process Context Switching – Definition

Process Context Switching is the process by which the operating system saves the context (state) of the currently running process and restores the context of another process, allowing the CPU to switch execution from one process to another.

Simple Definition (Exam)

Context Switching is the process of saving the state of the current process and loading the state of the next process so that the CPU can switch from one process to another.

What is saved during Context Switching?

- Program Counter (PC)
- CPU Registers
- Stack Pointer (SP)
- Process State
- Other CPU-related information

These are saved in the Process Control Block (PCB).

State Transition

```
Running Process A
       |
Save Context (PCB)
       |
Load Context of Process B
       ▼
Running Process B
```

One-line exam definition:

Context Switching is the process of saving the execution context of one process and restoring the execution context of another process to enable CPU sharing.

contex switching beshi hole kii hobe and kom hole ki hobe ?

Context Switching যত বেশি বা কম হবে, তার সরাসরি প্রভাব CPU Performance-এর উপর পড়ে।

---

1. Context Switching বেশি হলে

কী হয়?

- ❌ CPU-এর অনেক সময় Context Save ও Context Restore করতে নষ্ট হয়।
- ❌ কম সময় Process-এর আসল কাজ (Execution) হয়।
- ❌ CPU Overhead বেড়ে যায়।
- ❌ System Performance কমে যায়।
- ❌ Throughput কমে যায় (কম সংখ্যক process সম্পন্ন হয়)।
- ❌ Response Time কিছু ক্ষেত্রে বেড়ে যেতে পারে।

উদাহরণ

ধরো,

- একটি Context Switch = 5 µs
- ১ সেকেন্ডে 100,000 বার Context Switch হচ্ছে।

তাহলে CPU-এর অনেক সময় শুধু Process বদলাতেই চলে যাবে, কাজ করার সময় কমে যাবে।

---

2. Context Switching কম হলে

কী হয়?

- ✅ CPU বেশি সময় Process Execute করতে পারে।
- ✅ CPU Overhead কম হয়।
- ✅ System Performance ভালো হয়।
- ✅ Throughput বৃদ্ধি পায়।
- ✅ CPU Efficiency বাড়ে।

কিন্তু...

যদি Context Switching অত্যন্ত কম হয়,

তাহলে—

- একটি Process অনেকক্ষণ CPU দখল করে রাখতে পারে।
- অন্য Ready Process-গুলোকে বেশি অপেক্ষা করতে হয়।
- Response Time খারাপ হতে পারে।
- Interactive System-এ User Experience খারাপ হতে পারে।

Comparison

More Context Switching	Less Context Switching
High CPU overhead	Low CPU overhead
Lower CPU efficiency	Higher CPU efficiency
Lower system performance	Better system performance
Less execution time for processes	More execution time for processes
Too frequent process switching	Longer CPU usage by each process

---

Exam Answer (2 Marks)

More Context Switching

- Increases CPU overhead.
- Reduces system performance.
- Decreases CPU efficiency.

Less Context Switching

- Reduces CPU overhead.
- Improves CPU efficiency.
- Increases overall system performance.

One-line conclusion

Frequent context switching wastes CPU time due to excessive overhead, while an appropriate amount of context switching improves overall system performance and responsiveness.

contex switch er theke ki ki content ache sob contecnt golo amk bolo book theke ekdom

যদি তোমার Operating System বইতে Context Switching অধ্যায় থাকে, তাহলে সাধারণত নিচের টপিকগুলোই থাকে। এগুলো প্রায় সব স্ট্যান্ডার্ড OS বই (Silberschatz, Galvin, Stallings, Tanenbaum, xv6 lecture notes) এবং বিশ্ববিদ্যালয়ের কোর্সে পড়ানো হয়।

Context Switching Topics

1. Definition of Context Switching

- What is Context Switching?
- Why is Context Switching needed?

---

2. Process Context

- What is Process Context?
- Components of Process Context

---

3. Context (What is Saved?)

Context Switching-এর সময় কী কী Save হয়?

- Program Counter (PC)
- CPU Registers
- Stack Pointer (SP)
- Processor Status Word (PSW/Flags)
- Memory Management Information
- Process State
- Scheduling Information

---

4. Process Control Block (PCB)

- Definition
- Structure

- Why Context is saved in PCB

---

5. Steps of Context Switching

Typical sequence:

1. Interrupt/System Call
2. Save Current Process Context
3. Update PCB
4. Scheduler Runs
5. Select Next Process
6. Load New Process Context
7. Resume Execution

---

6. Scheduler

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

---

7. Dispatcher

- Definition
- Functions
- Dispatcher vs Scheduler
- Dispatch Latency

---

8. Descheduling

- Definition
- Running → Ready
- Running → Blocked

---

9. Scheduling Queue

- Job Queue
- Ready Queue
- Device Queue

---

10. Process State Transition

```
New
 ↓
```

```
Ready
 ↓
Running
 ↙     ↘
Blocked  Ready
 ↓
Ready
 ↓
Running
 ↓

Terminated
```

---

11. Causes of Context Switching

- Time Quantum Expired
- Higher Priority Process
- Interrupt
- I/O Request
- System Call
- Process Termination

---

12. Types of Context Switching

- Process Context Switch
- Thread Context Switch

---

13. Context Switch Overhead

- Saving Registers
- Loading Registers
- Cache Effects
- TLB Flush (in many systems)

---

14. Advantages

- Multitasking
- Better CPU Utilization
- Fair Scheduling
- Improved Responsiveness

---

15. Disadvantages

- CPU Overhead

- Performance Loss
- Cache Miss
- Increased Latency

---

16. Context Switch Time

- Definition
- Factors Affecting Context Switch Time

---

17. Context Switching vs Process Switching

Difference between

- Context Switch
- Process Switch

18. Context Switching vs Interrupt

Comparison

19. Context Switching vs Multitasking

Difference

পরীক্ষার জন্য সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন

1. Definition of Context Switching
2. Why Context Switching is needed?
3. What information is saved during Context Switching?
4. Explain the steps of Context Switching.
5. Context Switching using PCB.
6. Advantages and Disadvantages of Context Switching.
7. Scheduler and Dispatcher.
8. Context Switching vs Process Switching.
9. Context Switch Overhead.
10. Causes of Context Switching.

এই ১০টি টপিক থেকেই সাধারণত বিশ্ববিদ্যালয়ের Operating System পরীক্ষায় Context Switching সম্পর্কিত প্রায় সব প্রশ্ন আসে।

1. **Ultimate Operating System Exam Notes**
2. **📖 Chapter List**
3. **Chapter 1 : Introduction to Process**
4. Process
5. Program vs Process
6. Process Control Block (PCB)
7. Process ID (PID)

Chapter 2 : Process States

5. Embryo (New) State
6. Ready State
7. Scheduler
8. Running State
9. Descheduler
10. Blocked (Waiting) State
11. Zombie (Ending) State
12. Complete Process State Diagram

Chapter 3 : Process Suspension

13. Process Suspension
14. Suspend vs Terminate
15. Suspended Ready
16. Suspended Blocked
17. Swap Space
18. Page File
19. Sleep Mode

Chapter 4 : Context Switching

20. Process Context
21. Context Switching
22. Why Context Switching?
23. Steps of Context Switching
24. Information Saved
25. PCB and Context Switching

Preemptive Scheduling

Definition

Preemptive Scheduling is a CPU scheduling method in which the Operating System can interrupt a running process and allocate the CPU to another process before the current process completes its execution.

When is it Used?

- When a higher-priority process arrives.
- In **time-sharing systems**.
- In **interactive systems**, where quick response is required.
- When the CPU time quantum expires (e.g., in **Round Robin Scheduling**).

Example

P1 is currently using the CPU.
Suddenly, **P2** arrives with a higher priority.

The Operating System will:

1. Save the context of **P1**.
2. Allocate the CPU to **P2**.
3. After **P2** finishes or releases the CPU, **P1** resumes execution.

Advantages

- Fast response time.
- Better CPU utilization.
- High-priority processes are executed quickly.
- Supports time-sharing systems.

Disadvantages

- More context switching occurs.
- Higher CPU overhead.
- More complex scheduler implementation.

Examples of Preemptive Scheduling Algorithms

- Round Robin (RR)
- Shortest Remaining Time First (SRTF)
- Preemptive Priority Scheduling

Non-Preemptive Scheduling

Definition

**Non-Preemptive Scheduling is a CPU scheduling method in which a process keeps the CPU until it finishes
its execution or requests an I/O operation.
The Operating System cannot take the CPU away from a running process.**

When is it Used?

- In **batch processing systems**.
- Where response time is not critical.
- In simple operating systems.
- When minimizing context switching is important.

Example

Suppose:

- **P1** is using the CPU.
- Then **P2** arrives.

In this case:

- **P2** must wait.
- **P1** continues running until it finishes or blocks for I/O.
- Only then does **P2** get the CPU.

Advantages

- Less context switching.
- Lower CPU overhead.
- Simple implementation.
- Less CPU time is wasted.

Disadvantages

- High-priority processes may have to wait.
- Longer response time.
- Long-running processes may delay other processes (**Convoy Effect**).

Examples of Non-Preemptive Scheduling Algorithms

- First Come First Served (FCFS)
- Non-Preemptive Priority Scheduling
- Non-Preemptive Shortest Job First (SJF)

Preemptive vs Non-Preemptive

Feature	Preemptive	Non-Preemptive
CPU can be taken away from a running process	✅ Yes	❌ No
Context Switching	More	Less
Response Time	Faster	Slower

Complexity	Higher	Lower
Best For	Interactive, Real-Time, Time-Sharing Systems	Batch Processing Systems
Examples	RR, SRTF, Preemptive Priority	FCFS, SJF, Non-Preemptive Priority

Easy Way to Remember

- **Preemptive = The Operating System can interrupt a running process and take the CPU away.**
- **Non-Preemptive = A process keeps the CPU until it finishes execution or voluntarily releases it.**

17.

🖥️ Ultimate Operating System Exam Notes - Complete Chapter by Chapter

18.
19. ---
20.
21. ## 📖 CHAPTER 1: INTRODUCTION TO PROCESS
22.
23. ---
24.
25. ### 1. What is a Process?
26.
27. **Definition:**

A process is a program in execution. It is an active entity that includes the program code, current activity, stack, data section, and process control block.

Program vs Process:

Program	Process
Passive entity	Active entity
Stored on disk	Loaded in memory
Static	Dynamic
Contains only code	Contains code, data, stack, PCB
No execution state	Has execution state

Process Components:

1. **Text Section** - Program code
2. **Data Section** - Global variables
3. **Heap** - Dynamically allocated memory
4. **Stack** - Temporary data (function parameters, return addresses)
5. **Program Counter (PC)** - Next instruction address

---

2. Process ID (PID)

Definition:

A Process ID (PID) is a unique identification number assigned by the Operating System to every running process for identification and management.

Example:

Process Name	Process ID (PID)
chrome.exe	2548
notepad.exe	7352
explorer.exe	2456

Importance:

✅ **Process Identification** - Uniquely identifies each process
✅ **Process Management** - Allows OS to monitor and manage processes
✅ **Process Termination** - Enables killing specific processes using PID
✅ **Parent-Child Relationship** - Maintains relationships between processes

Key Points:

- No two running processes have the same PID
- Once terminated, PID may be reused

- PID is stored in the Process Control Block (PCB)

---

3. Process Control Block (PCB)

Definition:
A Process Control Block (PCB) is a data structure maintained by the Operating System that contains all important information about a process.

Components of PCB:

```
┌─────────────────────────────────────────┐
│        PROCESS CONTROL BLOCK            │
├─────────────────────────────────────────┤
│  1. Process ID (PID)                    │
│  2. Process State                       │
│  3. Program Counter (PC)                │
│  4. CPU Registers                       │
│  5. Stack Pointer                       │
│  6. Memory Management Info              │
│  7. Scheduling Information              │
│  8. I/O Status Information              │
│  9. List of Open Files                  │
│ 10. Accounting Information              │
└─────────────────────────────────────────┘
```

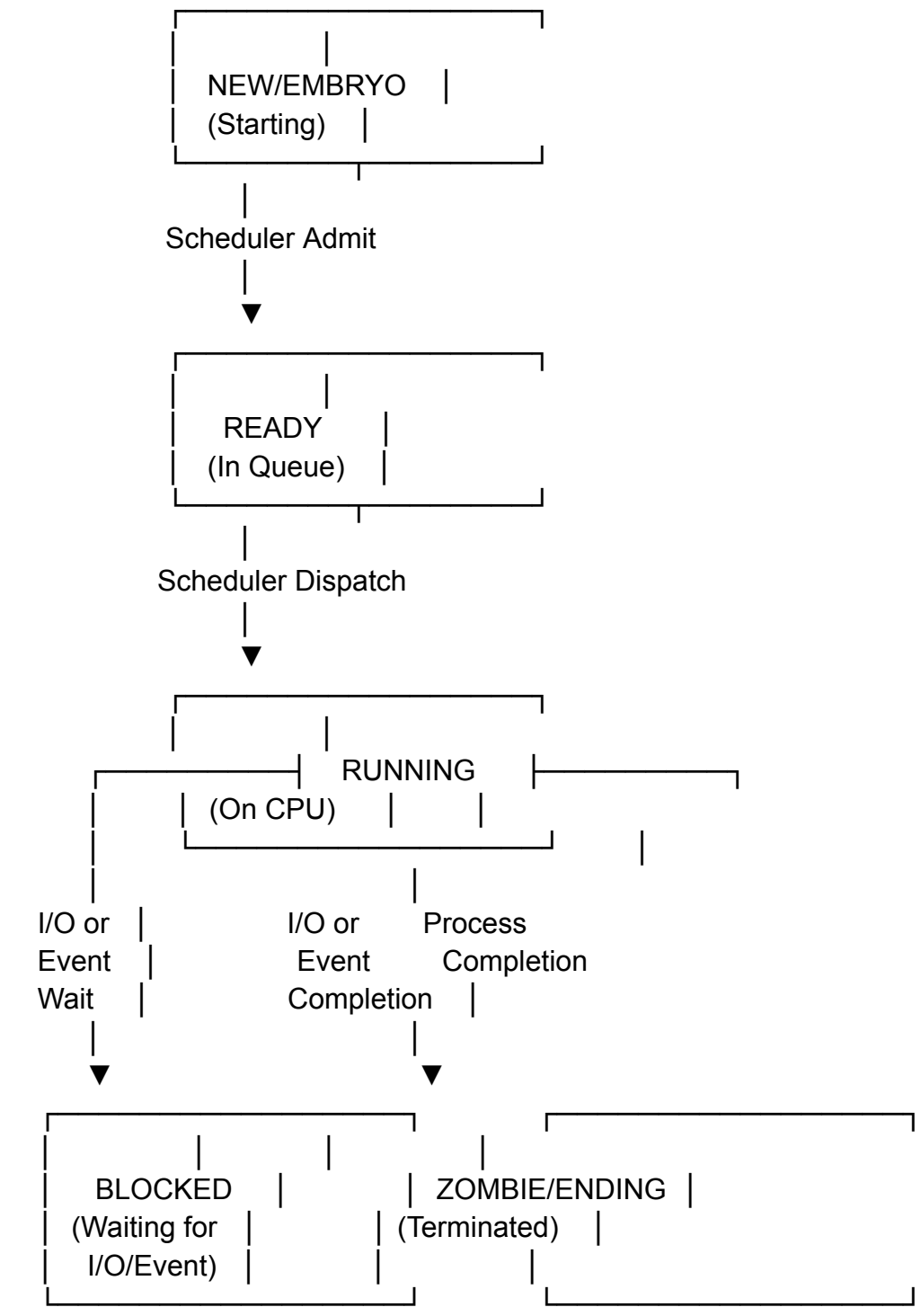
Location:
```
RAM (Main Memory)
│
└── Kernel Space
      │
      └── Process Control Block (PCB)
```

Why PCB is Important:
- Contains all process information
- Allows context switching
- Enables process management
- Stored in kernel memory

📖 CHAPTER 2: PROCESS STATES

---

4. Process State Diagram



---

5. Embryo (New) State

Definition:
The Embryo State is the initial state of a process when it is being created by the Operating System.

What happens in Embryo State:
✅ The process has already been created
✅ A PID has been assigned
✅ The PCB has been created
✅ Memory has been allocated to the process
✅ Required resources are allocated
✅ Process is ready to enter Ready Queue

Process Creation Steps:
```
1. Process Creation Request
        |
        ▼
2. PID Assigned
        |
        ▼
3. PCB Created
        |
        ▼
4. Memory Allocated
        |
        ▼
5. Resources Allocated
        |
        ▼
6. Ready Queue Entry
```

Simple Memory:
Embryo/New State = **Process Creation Phase**
- PID তৈরি
- PCB তৈরি
- Memory Allocate
- Resource Allocate

---

6. Ready State

Definition:
The Ready State is the state in which a process is fully prepared to execute but is waiting for the CPU Scheduler to assign the CPU.

What happens in Ready State:
✅ Process is placed in the **Ready Queue**
✅ Process is waiting for the **CPU Scheduler**
✅ All necessary resources are allocated
❌ Process is **NOT using the CPU**
❌ No instructions are being executed
✅ Process will move to Running State when CPU is assigned

Transitions:
```
Embryo → Ready  (When created)
Running → Ready  (When time quantum expires)
Blocked → Ready (When I/O completes)
```

---

7. Scheduler

Definition:
A Scheduler is an Operating System component that selects which process from the Ready Queue will be assigned to the CPU.

Types of Schedulers:
Scheduler Type	Purpose	Frequency
Long-Term Scheduler	Selects processes to be loaded into memory	Rarely
Short-Term Scheduler	Selects process from Ready Queue for CPU	Very frequently
Medium-Term Scheduler	Removes/swaps processes from memory	Moderate

Short-Term Scheduler:
```
Ready Queue
```

```
    │
    ▼
Short-Term Scheduler
    │
    ▼
Selects Next Process
    │
    ▼
CPU Assigned (Running)
```

---

8. Running State

Definition:
Running State is the state in which a process is actively executing instructions using the CPU.

What happens in Running State:
✅ Process is assigned the CPU by Scheduler
✅ CPU executes process instructions
✅ Program Counter (PC) continuously updates
✅ CPU register values are continuously updated
✅ Process may access memory, files, and I/O devices
✅ Process performs computations and executes tasks

CPU Status:
✅ Process is using the CPU
✅ Instructions are being executed
❌ Process is NOT waiting for CPU

Transitions from Running State:
```
Running
   ├──→ Ready (Time slice expires - Preemption)
   ├──→ Blocked (I/O Request)
   └──→ Zombie (Process completes execution)
```

---

9. Descheduler

Definition:
A Descheduler is a component that removes a running process from the CPU and places it back into the Ready Queue or Blocked State.

When Descheduling Happens:
1. **Time Quantum Expires** - Running → Ready
2. **Higher Priority Process** - Running → Ready (Preemption)
3. **I/O Request** - Running → Blocked
4. **Process Completion** - Running → Zombie

Descheduling Process:
```
Running Process
      |
      ▼
Save Context to PCB
      |
      ▼
Update Process State
      |
      ▼
Move to Appropriate Queue
      |
      ▼
Scheduler Selects Next Process
```

---

10. Blocked (Waiting) State

Definition:
The Blocked State is the state in which a process is waiting for some event (I/O completion, resource availability, etc.) and cannot continue execution.

What happens in Blocked State:
✅ Process is waiting for an event
❌ CPU is not used
❌ No instructions are executed
✅ Process remains in memory
✅ Will move to Ready when event completes

Common Causes of Blocking:
1. **I/O Operations** - Reading from disk, keyboard, network

2. **Resource Unavailability** - Waiting for mutex, semaphore
3. **Event Waiting** - Waiting for timer, signal

Transition:
```
Running → Blocked (I/O Request)
Blocked → Ready (I/O Complete)
```

---

11. Zombie (Ending) State

Definition:
Zombie State is the state in which a process has completed its execution, but its Process Control Block (PCB) and exit status remain in the Operating System until the parent process collects them.

What happens in Zombie State:
✅ Process has completed execution
❌ Process does NOT use the CPU
❌ No instructions are executed
✅ Most memory (code, data, stack, heap) is released
✅ Only PCB and exit status remain
✅ OS waits for parent to collect exit status

Memory Status:
```
Memory Status in Zombie:
    ├── Code Section → Released
    ├── Data Section → Released
    ├── Stack → Released
    ├── Heap → Released
    └── PCB → Remains in Kernel
```

Why Zombie Exists:
- Parent process hasn't called `wait()` or `waitpid()`
- After parent collects info, process is fully destroyed

Transition:
```
Running
   │
```

```
    ▼
Zombie (Ending)
    │
Parent collects exit status
    │
    ▼
Destroyed (Completely Removed)
```

Exam Definition:
Zombie State is the state in which a process has completed execution, but its PCB and exit status remain in the Operating System until the parent process collects them.

📖 CHAPTER 3: PROCESS SUSPENSION

---

12. Process Suspension

Definition:
Process Suspension is the temporary stopping of a process without terminating it, while preserving its execution state so that it can be resumed later.

Process State Transition:
```
Running
   ↓
Suspending
   ↓
Suspended
```

Internal Operations during Suspension:

A. State Change:
The process state changes to either:
- Suspended Ready
- Suspended Blocked

B. Resource and CPU Release:

- Process removed from CPU execution
- CPU usage becomes 0%
- CPU time becomes available for other processes

C. Context Saving:
The OS saves the complete execution context:
- CPU Registers
- Program Counter (PC)
- Stack Pointer
- Stack
- Other processor state information

---

13. Suspend vs Terminate

Feature	Suspend	Terminate
Duration	Temporarily stops	Permanently ends
Resume	Can be resumed later	Cannot be resumed
State	Preserved	Destroyed
CPU Usage	Becomes 0%	Completely released
Resources	Held (or swapped)	All released
PCB	Retained	Destroyed

---

14. Suspended Ready vs Suspended Blocked

Suspended Ready:
- Process was in **Ready State**
- Suspended by OS
- Not ready to execute
- In memory or swapped to disk
- Can be resumed → Ready

Suspended Blocked:
- Process was in **Blocked State**
- Suspended by OS
- Waiting for I/O
- May be swapped to disk
- Event completion → Suspended Ready

Diagram:

```
Running → Suspending → Suspended Ready
Running → Suspending → Suspended Blocked

Suspended Ready → Ready (After Resume)
Suspended Blocked → Suspended Ready (After I/O Complete)
```

---

15. Swap Space

Definition:
Swap Space is a dedicated area on secondary storage (disk) where the Operating System moves processes when RAM is insufficient.

Memory Swapping:
```
RAM (Main Memory)
    │
    ├── Kernel Space (PCB stays here)
    │
    └── User Space (Process Memory)
          │
          │  (When RAM is full)
          ▼
    Secondary Storage
          │
          └── Swap Space (Linux)
          │
          └── Pagefile (Windows)
```

Key Points:
- PCB remains in RAM
- Only process memory (code, data, stack, heap) may be moved
- Swapping helps manage memory efficiently

Why Swap Happens:
1. Memory is low
2. Less important processes
3. Background processes
4. Power saving

---

16. Sleep Mode

Definition:
Sleep Mode is a practical example of process suspension where all processes are suspended, and the computer enters a low-power state.

What happens in Sleep Mode:
- Running processes are suspended
- Execution state remains in RAM
- Computer enters low-power state
- When woken, processes resume from where they stopped

Memory Management:
```
Normal Operation:
    RAM (Active) ← All processes

Sleep Mode:
    RAM (Low Power) ← Process states preserved

Wake Up:
    RAM (Active) ← Process states restored
```

📖 CHAPTER 4: CONTEXT SWITCHING

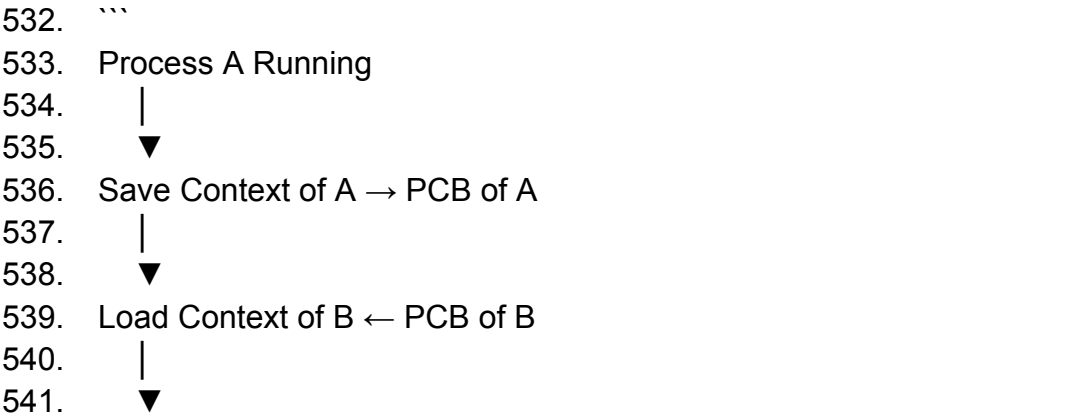
---

17. Context Switching - Definition

Definition:
Context Switching is the process of saving the execution context of one process and restoring the execution context of another process to enable CPU sharing.

One-Line Definition:
Context Switching is the process of saving the state of the current process and loading the state of the next process so that the CPU can switch from one process to another.

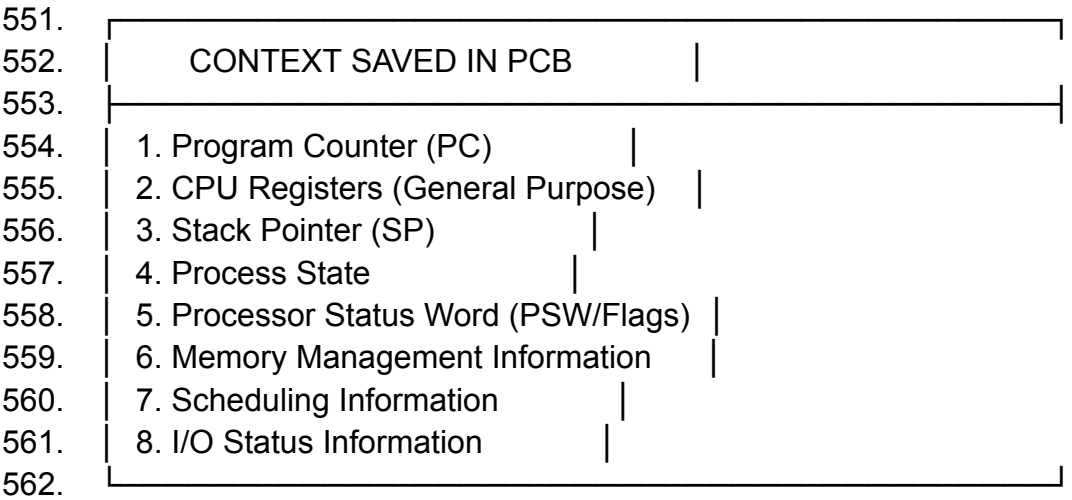
Context Switch Visualization:



---

18. What is Saved During Context Switching?

Information Saved in PCB:



Why Save All This?
- Allows process to continue from the exact same point
- Preserves all CPU-related information
- Enables seamless switching between processes

---

19. Steps of Context Switching

Complete Context Switching Process:

```
Step 1: Interrupt or System Call Occurs
        |
        ▼
Step 2: Save Context of Current Process
      - Save PC to PCB
      - Save Registers to PCB
      - Save Stack Pointer to PCB
      - Save Process State
        |
        ▼
Step 3: Update PCB of Current Process
      - Change State: Running → Ready/Blocked
      - Update Scheduling Information
        |
        ▼
Step 4: Move Process to Appropriate Queue
      - Ready Queue OR Blocked Queue
        |
        ▼
Step 5: Short-Term Scheduler Runs
      - Select Next Process from Ready Queue
        |
        ▼
Step 6: Load Context of New Process
      - Load PC from PCB
      - Load Registers from PCB
      - Load Stack Pointer from PCB
        |
        ▼
Step 7: Resume Execution of New Process
```

Simple Flow:
```
Save Context → Update PCB → Select Next Process → Load Context → Resume
```

---

20. Context Switching Components

A. Context (What is Saved?)
```
Program Counter (PC) - Next instruction address
CPU Registers - General purpose registers
Stack Pointer (SP) - Top of stack
Processor Status Word (PSW) - Flags and status
Memory Management Info - Page table base, TLB
Scheduling Info - Priority, time quantum
Process State - Current state (Ready, Running, etc.)
```

B. Process Control Block (PCB)
```
┌──────────────────────────────────────────┐
│         PROCESS CONTROL BLOCK            │
├──────────────────────────────────────────┤
│ PID                                      │
│ Process State                            │
│ Program Counter                          │
│ CPU Registers                            │
│ Stack Pointer                            │
│ Memory Management Info                   │
│ Scheduling Info                          │
│ I/O Status Info                          │
│ List of Open Files                       │
└──────────────────────────────────────────┘
```

C. Scheduler
```
Types:
1. Long-Term Scheduler  - Loads processes into memory
2. Short-Term Scheduler - Selects process for CPU
3. Medium-Term Scheduler - Swaps processes in/out
```

D. Dispatcher
```
Functions:
1. Context Switching
2. Switching to User Mode
3. Jumping to Proper Location
```

Dispatcher vs Scheduler:

Scheduler	Dispatcher
Selects which process to run	Performs the actual switch
Decision making	Action execution
Runs less frequently	Runs frequently
Part of scheduling algorithm	Low-level mechanism

Dispatch Latency:
- Time taken by dispatcher to stop one process and start another
- Should be as low as possible

---

21. Causes of Context Switching

1. Time Quantum Expires
- Process has used its CPU time slice
- Preemptive scheduling

2. Higher Priority Process Arrives
- New process with higher priority
- Preemptive scheduling

3. I/O Request
- Process needs I/O operation
- Can't continue without it

4. Interrupt
- Hardware or software interrupt
- CPU needs to handle it

5. System Call
- Process requests OS service
- May require context switch

6. Process Termination
- Process completes execution
- OS needs to schedule next

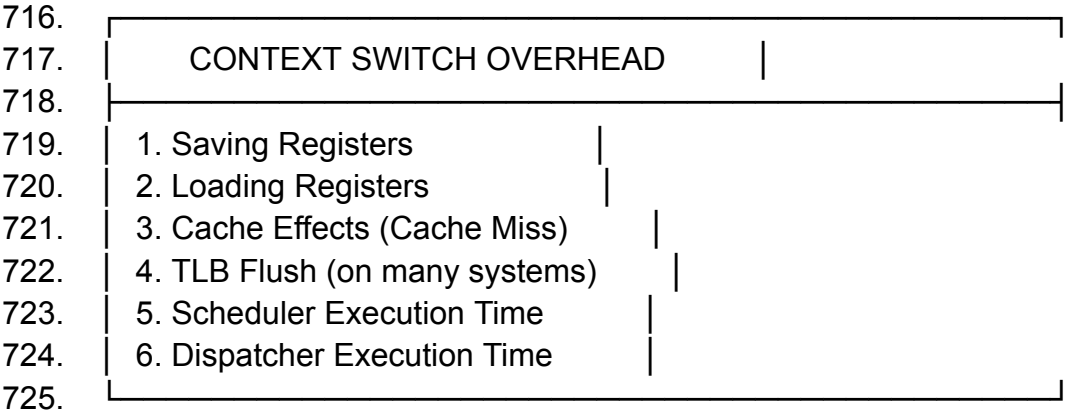
7. Voluntary Yield
- Process voluntarily releases CPU
- Non-preemptive scheduling

---

22. Context Switching Overhead

Definition:
Context Switching Overhead refers to the time and resources consumed by the OS when performing a context switch.

Components of Overhead:



Factors Affecting Overhead:
1. Number of registers to save/restore
2. Processor speed
3. Cache size
4. TLB implementation
5. Scheduling algorithm complexity
6. Operating system design

Overhead Example:
```
If Context Switch Time = 5 μs
Process Time Quantum = 100 ms
Overhead = 5/100000 = 0.005% (very small)

But if Context Switch = 5 μs
Time Quantum = 1 ms
Overhead = 5/1000 = 0.5% (significant)
```

---

23. More vs Less Context Switching

More Context Switching:
Effect	Description
❌ High CPU Overhead	Much time wasted saving/restoring contexts
❌ Lower CPU Efficiency	Less time for actual execution
❌ Poor System Performance	Throughput decreases
❌ Less Execution Time	Each process gets less CPU time
✅ Better Responsiveness	Interactive systems respond faster
✅ Faster Response Time	Quicker reaction to user input

Less Context Switching:
Effect	Description
✅ Low CPU Overhead	More time for execution
✅ Higher CPU Efficiency	Better throughput
✅ Better System Performance	More processes completed
✅ More Execution Time	Each process gets more CPU time
❌ Poor Responsiveness	Processes may starve
❌ Higher Response Time	Users may experience delays

Comparison Table:
Feature	More Context Switching	Less Context Switching
CPU Overhead	High	Low
CPU Efficiency	Low	High
System Performance	Lower	Better
Response Time	Better	Worse
Process Starvation	Less likely	More likely
Throughput	Lower	Higher

Exam Conclusion:
> Frequent context switching wastes CPU time due to excessive overhead, while an appropriate amount of context switching improves overall system performance and responsiveness.

---

24. Advantages and Disadvantages of Context Switching

Advantages:
```
┌──────────────────────────────────────────────────────────────┐
```

```
┌──────────────────────────────────────────────────┐
│      ADVANTAGES OF CONTEXT SWITCHING             │
├──────────────────────────────────────────────────┤
│ ✅ Enables Multitasking                          │
│   - Multiple processes share CPU                 │
│                                                  │
│ ✅ Better CPU Utilization                        │
│   - CPU never idle for long                      │
│                                                  │
│ ✅ Fair Scheduling                               │
│   - All processes get CPU time                   │
│                                                  │
│ ✅ Improved Responsiveness                       │
│   - Interactive systems work well                │
│                                                  │
│ ✅ Supports Time-Sharing                         │
│   - Multiple users can use system                │
│                                                  │
│ ✅ High Priority Process Handling                │
│   - Important tasks get priority                 │
└──────────────────────────────────────────────────┘
```

Disadvantages:

```
┌──────────────────────────────────────────────────┐
│    DISADVANTAGES OF CONTEXT SWITCHING            │
├──────────────────────────────────────────────────┤
│ ❌ CPU Overhead                                  │
│   - Time wasted on switching                     │
│                                                  │
│ ❌ Cache Misses                                  │
│   - Cache gets flushed/invalidated               │
│                                                  │
│ ❌ TLB Flush (in many systems)                   │
│   - Address translation cache cleared            │
│                                                  │
│ ❌ Performance Loss                              │
│   - Overall system performance drops             │
│                                                  │
│ ❌ Increased Latency                             │
│   - Switching takes time                         │
│                                                  │
│ ❌ Complex Implementation                        │
│   - OS needs sophisticated scheduler             │
```

└──┘
```

---

### 25. Context Switch Time

**Definition:**
Context Switch Time is the time required for the Operating System to save the context of one process and load the context of another process.

**Context Switch Time Calculation:**
```
Context Switch Time = Save Time + Load Time + Scheduler Time

Where:
- Save Time = Time to save registers, PC, SP, etc.
- Load Time = Time to load registers, PC, SP, etc.
- Scheduler Time = Time for scheduler to select next process
```

**Factors Affecting Context Switch Time:**

```
1. Number of Registers
   └── More registers → More time

2. CPU Speed
   └── Faster CPU → Less time

3. Memory Speed
   └── Faster RAM → Less time

4. Cache Size
   └── Larger cache → Less time for TLB

5. TLB Implementation
   └── Hardware TLB → Less time

6. OS Implementation
   └── Efficient OS → Less time

7. Process Size
   └── Larger process → More time (if memory info saved)
```
```

```
Simple System: 1-5 microseconds
Complex System: 10-50 microseconds
Worst Case: Up to 100+ microseconds
```

Impact on System:
```
If Context Switch = 10 µs
Time Quantum = 100 ms
Overhead = 0.01%

If Context Switch = 10 µs
Time Quantum = 1 ms
Overhead = 1%
```

📖 CHAPTER 5: SCHEDULING (PREEMPTIVE vs NON-PREEMPTIVE)

---

26. Preemptive Scheduling

Definition:
Preemptive Scheduling is a CPU scheduling method in which the Operating System can interrupt a running process and allocate the CPU to another process before the current process completes its execution.

When Preemption Occurs:
```
1. Higher Priority Process Arrives
   └── OS interrupts current process

2. CPU Time Quantum Expires
   └── Round Robin Scheduling

3. System Call or Interrupt
   └── OS takes control
```

4. I/O Request
   └── Process voluntarily gives up CPU
```

**Example:**
```
P1 is using CPU (Time: 0-10 ms)
P2 arrives (Higher Priority) at 5 ms
        │
        ▼
OS Saves Context of P1
        │
        ▼
OS Loads Context of P2
        │
        ▼
P2 Runs (Time: 5-15 ms)
        │
        ▼
P2 Finishes or Releases CPU
        │
        ▼
P1 Resumes from where it stopped
```

**Advantages:**
```
✅ Fast Response Time
   └── Interactive users get quick feedback

✅ Better CPU Utilization
   └── CPU is always busy

✅ High Priority Processes Execute Quickly
   └── Critical tasks get CPU immediately

✅ Supports Time-Sharing Systems
   └── Multiple users share CPU fairly
```

**Disadvantages:**
```
❌ More Context Switching

```
   └── Higher CPU overhead

❌ Higher CPU Overhead
   └── Time wasted on switching

❌ Complex Scheduler Implementation
   └── OS code is more complicated
```

Preemptive Scheduling Algorithms:
```
1. Round Robin (RR)
   └── Each process gets fixed time quantum

2. Shortest Remaining Time First (SRTF)
   └── Process with shortest remaining time gets CPU

3. Preemptive Priority Scheduling
   └── Higher priority process preempts lower priority
```

---

27. Non-Preemptive Scheduling

Definition:
Non-Preemptive Scheduling is a CPU scheduling method in which a process keeps the CPU until it finishes its execution or requests an I/O operation. The Operating System cannot take the CPU away from a running process.

When Process Releases CPU:
```
1. Process Completes Execution
   └── Naturally terminates

2. Process Requests I/O
   └── Voluntarily gives up CPU

3. Process Calls yield()
   └── Voluntarily releases CPU
```

Example:

```
P1 is using CPU (Time: 0-20 ms)
P2 arrives at 5 ms
P3 arrives at 10 ms
        |
        ▼
P1 continues running
        |
        ▼
P1 finishes at 20 ms
        |
        ▼
P2 gets CPU (Time: 20-40 ms)
```

Advantages:
```
✅ Less Context Switching
   └── Lower CPU overhead

✅ Lower CPU Overhead
   └── More time for execution

✅ Simple Implementation
   └── Scheduler is easy to design

✅ Less CPU Time Wasted
   └── No unnecessary switches
```

Disadvantages:
```
❌ High Priority Processes May Wait
   └── Critical tasks get delayed

❌ Longer Response Time
   └── Interactive systems suffer

❌ Convoy Effect
   └── Long processes delay short ones

❌ Process Starvation Possible
   └── Some processes may wait indefinitely
```

Non-Preemptive Scheduling Algorithms:
```
1. First Come First Served (FCFS)
   └── Process that arrives first gets CPU

2. Non-Preemptive Priority Scheduling
   └── Higher priority process gets CPU

3. Shortest Job First (SJF)
   └── Shortest process gets CPU first
```

---

28. Preemptive vs Non-Preemptive - Complete Comparison

Feature	Preemptive	Non-Preemptive
CPU Interruption	✅ Can be taken away	❌ Cannot be taken
Context Switching	More frequent	Less frequent
CPU Overhead	Higher	Lower
Response Time	Faster	Slower
Complexity	Higher	Lower
CPU Utilization	Better	Lower
Throughput	Variable	Generally higher
Fairness	More fair	Less fair
Best For	Interactive, Real-Time	Batch Processing
Process Starvation	Less likely	More likely
Convoy Effect	Less common	More common
Examples	RR, SRTF, Priority	FCFS, SJF, Non-Preemptive Priority

Easy Way to Remember:
```
Preemptive = OS Can Interrupt Running Process
Non-Preemptive = Process Keeps CPU Until Done

Preemptive = "I can take CPU from you"
Non-Preemptive = "You keep CPU until you're done"
```

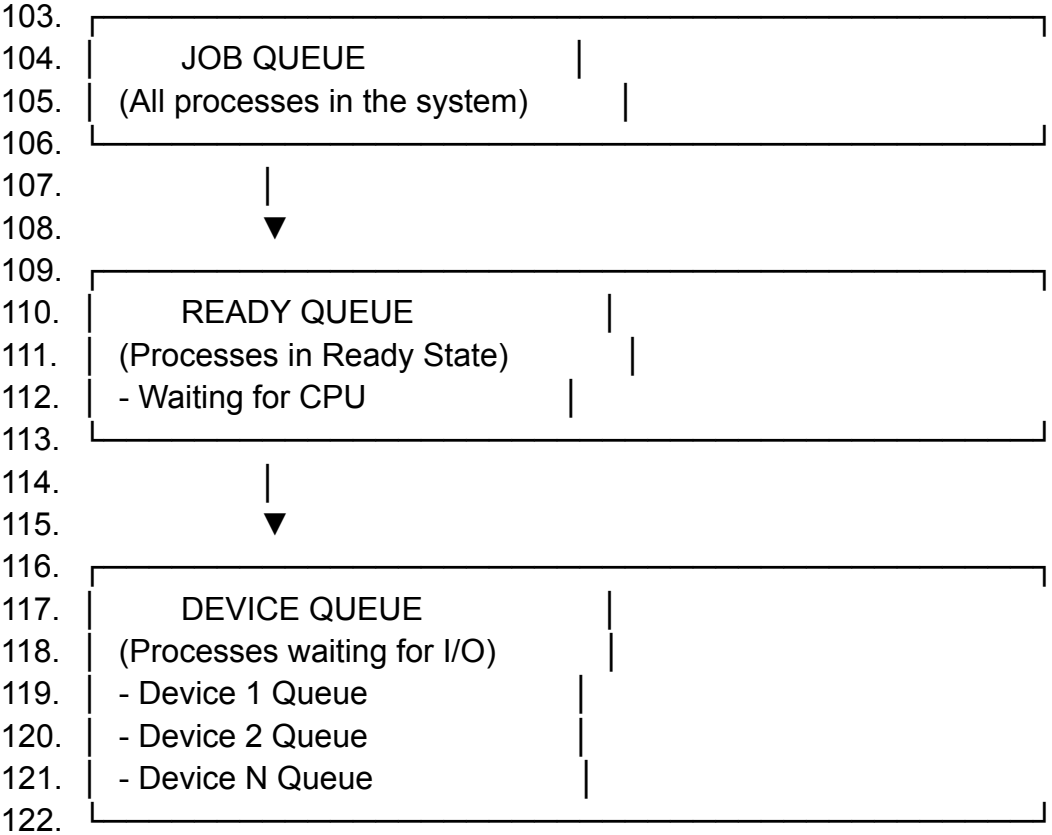
📖 CHAPTER 6: SCHEDULING QUEUES

---

29. Process Scheduling Queues

Definition:
Process Scheduling Queues are data structures maintained by the Operating System to hold processes in different states.

Queue Types:



Queue Components:

```
1. Job Queue
   ├── All processes in the system
   ├── Includes new, ready, running, blocked
   └── Maintained in secondary storage

2. Ready Queue
   ├── Processes in Ready State
```

```
    ├── Waiting for CPU allocation
    └── Implemented as linked list or array

3. Device Queue (I/O Queue)
    ├── Processes waiting for I/O device
    ├── One queue per device
    └── Example: Disk Queue, Printer Queue
```

Queue Operations:

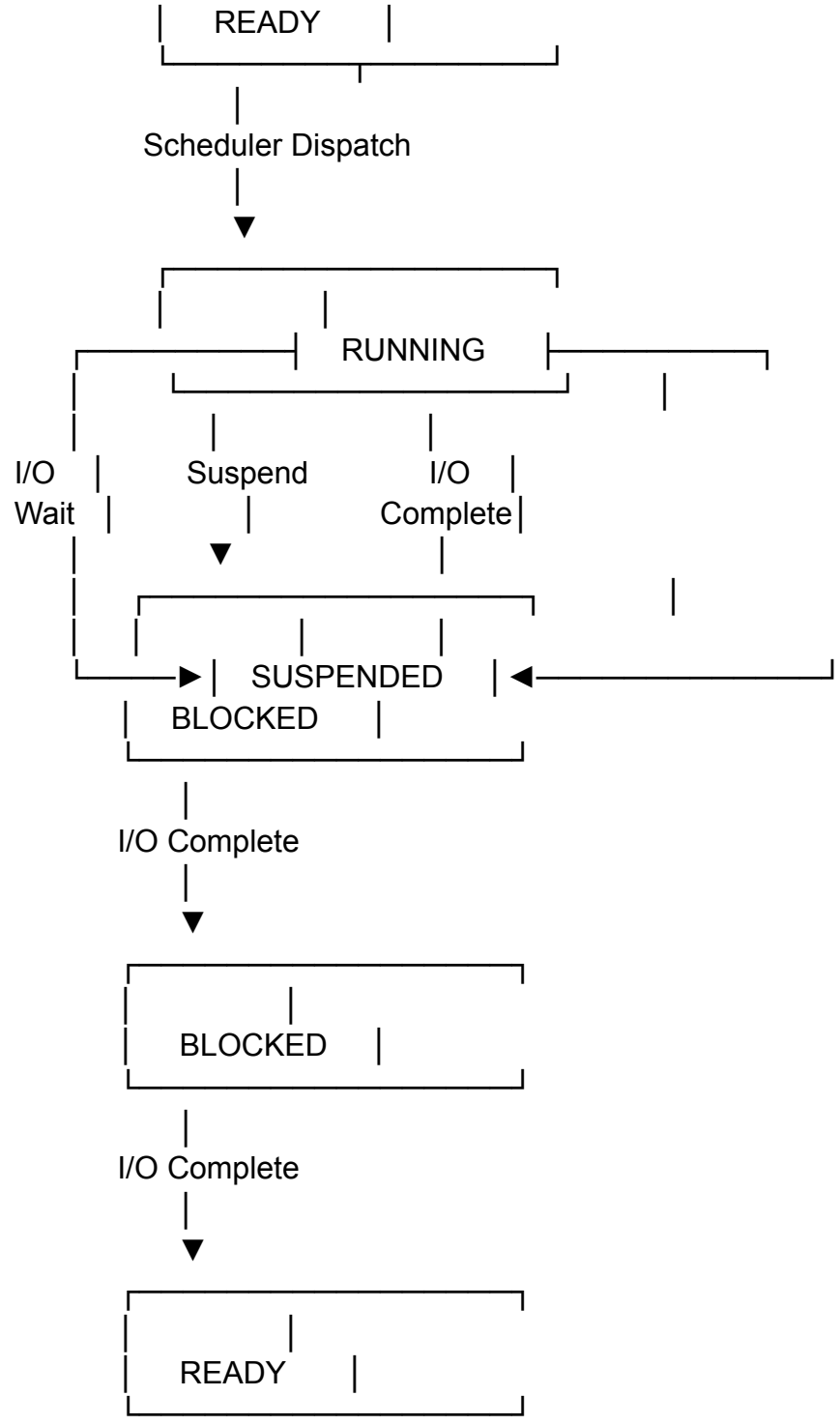
```
Enqueue: Add process to queue
Dequeue: Remove process from queue
Peek: View next process without removing
```

📖 CHAPTER 7: KEY DIAGRAMS AND TABLES

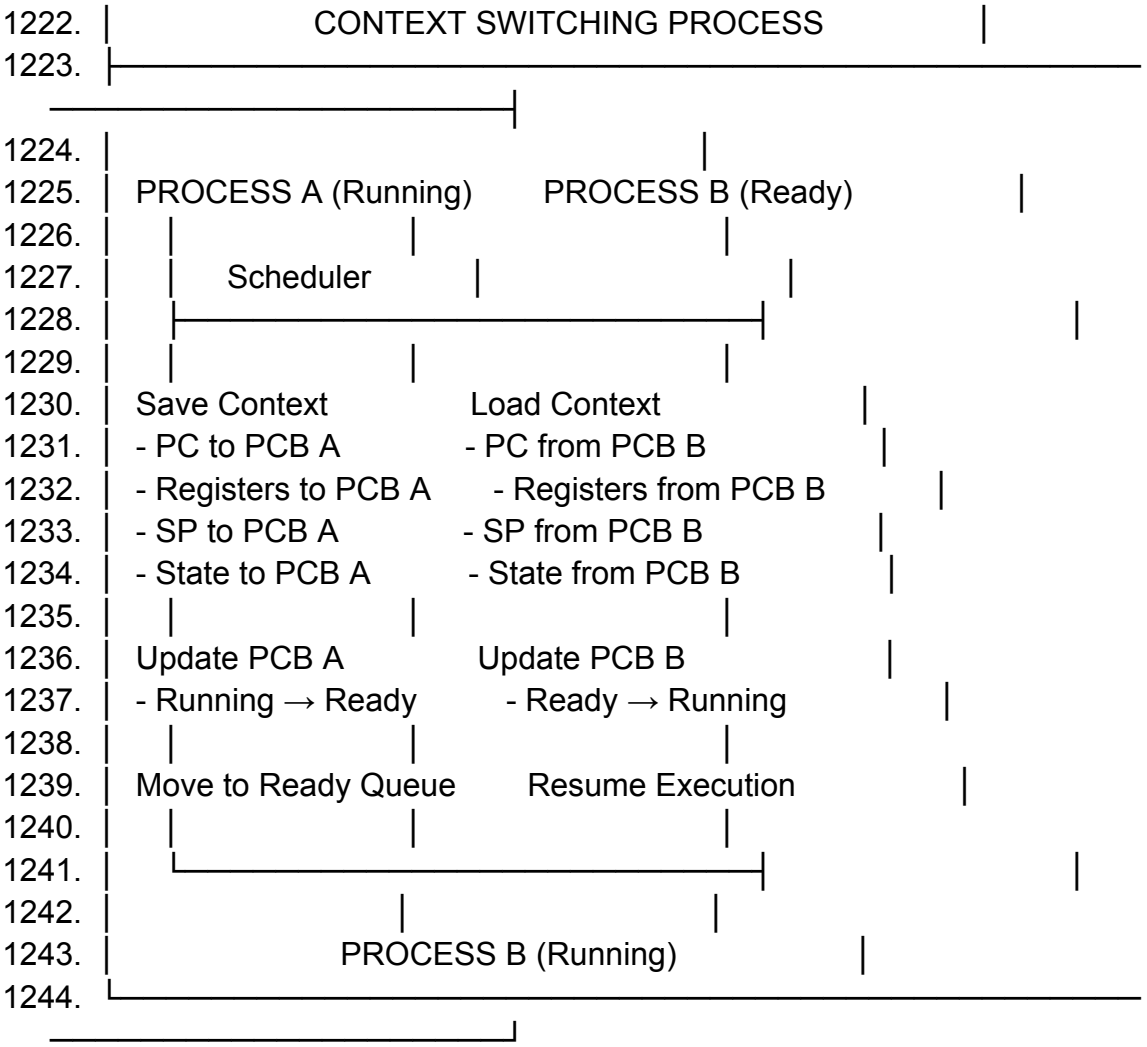
---

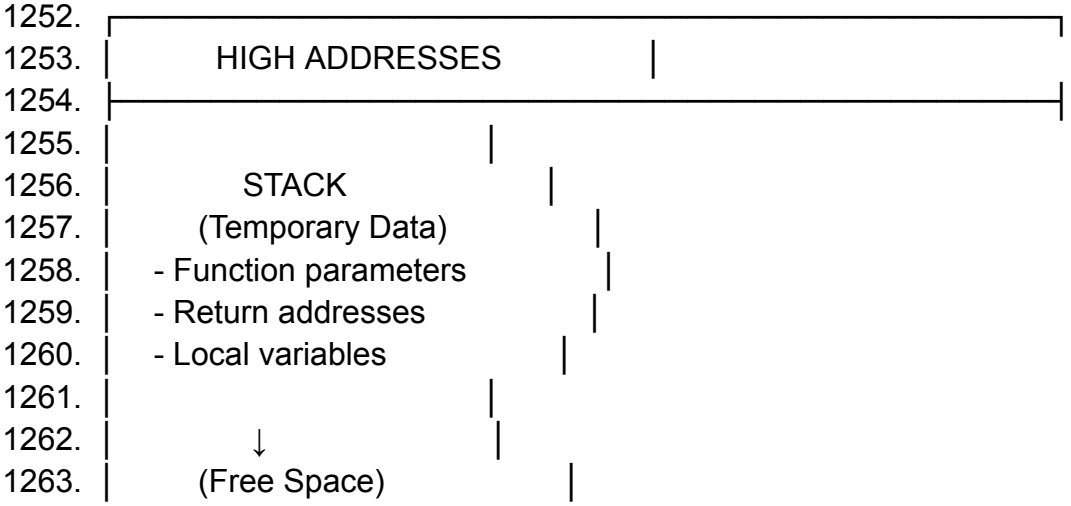
30. Complete Process State Diagram (With Suspension)

```
                ┌──────────────────────┐
                │                      │
                │   NEW/EMBRYO         │
                └──────────┬───────────┘
                           │
                    Scheduler Admit
                           │
                           ▼
                ┌──────────────────────┐
                │                      │
                │   SUSPENDED          │ ←──── Suspended Ready
                │    READY             │
                └──────────┬───────────┘
                           │
                        Resume
                           │
                           ▼
                ┌──────────────────────┐
                │                      │
```




```

---

### 31. Context Switching Complete Diagram

```
┌──
──────────────────────────┐




```

---

### 32. Memory Layout of a Process

```


```
```

```
┌─────────────────────────────────────────┐
│              ↑                          │
│                                         │
│             HEAP                        │
│        (Dynamic Memory)                 │
│    - malloc()/new allocations           │
│                                         │
├─────────────────────────────────────────┤
│          DATA SECTION                   │
│    - Global Variables                   │
│    - Static Variables                   │
│                                         │
├─────────────────────────────────────────┤
│          TEXT SECTION                   │
│    - Program Code                       │
│    - Instructions                       │
│                                         │
├─────────────────────────────────────────┤
│          LOW ADDRESSES                  │
└─────────────────────────────────────────┘
```

---

33. Scheduling Comparison Table

```
┌──────────────────────────────────────────────────────────────────────┐
│              PREEMPTIVE vs NON-PREEMPTIVE SCHEDULING                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Criteria          Preemptive          Non-Preemptive                │
│                                                                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  CPU Interruption  Can be interrupted   Cannot be interrupted        │
│                                                                      │
│  Context Switch    More frequent        Less frequent                │
│                                                                      │
│  CPU Overhead      Higher               Lower                        │
│                                                                      │
│  Response Time     Faster               Slower                       │
```

```
│                                                      │
│  Complexity       Higher             Lower              │
│                                                      │
│  CPU Utilization   Better             Lower              │
│                                                      │
│  Throughput       Variable           Generally higher       │
│                                                      │
│  Fairness         More fair          Less fair             │
│                                                      │
│  Starvation       Less likely        More likely            │
│                                                      │
│  Convoy Effect     Less common        More common             │
│                                                      │
│  Best For         Interactive        Batch Processing       │
│                Real-Time                              │
│                Time-Sharing                            │
│                                                      │
│  Examples         Round Robin        FCFS                 │
│                SRTF               SJF                   │
│                Priority (Preemp.)   Priority (Non-Preemp.) │
│                                                      │
└──────────────────────────────────────────────────────────────────────────────┘
```

📖 CHAPTER 8: EXAM QUESTIONS AND ANSWERS

---

34. 1 Mark Questions with Answers

Q1: What is a Process ID (PID)?
> A PID is a unique identification number assigned by the OS to every running process.

Q2: Define Context Switching.
> Context Switching is the process of saving the context of one process and loading the context of another.

Q3: What is PCB?
> PCB (Process Control Block) is a data structure containing all information about a process.

1344.
1345. **Q4: What is the Zombie State?**
1346. > Zombie State is when a process has completed execution but its PCB remains until parent collects exit status.
1347.
1348. **Q5: What is Preemptive Scheduling?**
1349. > Preemptive Scheduling allows the OS to interrupt a running process and assign CPU to another.
1350.
1351. **Q6: What is Non-Preemptive Scheduling?**
1352. > Non-Preemptive Scheduling is where a process keeps the CPU until it finishes or requests I/O.
1353.
1354. **Q7: What is Process Suspension?**
1355. > Process Suspension is temporarily stopping a process without terminating it.
1356.
1357. **Q8: What is a Ready Queue?**
1358. > Ready Queue contains processes in Ready State waiting for CPU allocation.
1359.
1360. **Q9: What is a Program Counter?**
1361. > Program Counter holds the address of the next instruction to be executed.
1362.
1363. **Q10: What is Swap Space?**
1364. > Swap Space is disk area where OS moves process memory when RAM is insufficient.
1365.
1366. ---
1367.
1368. ### 35. 2 Mark Questions with Answers
1369.
1370. **Q1: Explain the difference between Program and Process.**
1371. ```
1372. Program (Passive):
1373. - Stored on disk
1374. - Contains only code
1375. - Static entity
1376.
1377. Process (Active):
1378. - Loaded in memory
1379. - Contains code, data, stack, PCB
1380. - Dynamic entity

```
```

Q2: What information is stored in PCB?
```
1. Process ID (PID)
2. Process State
3. Program Counter (PC)
4. CPU Registers
5. Stack Pointer (SP)
6. Memory Management Info
7. Scheduling Info
8. I/O Status Info
```

Q3: Explain the Running State of a process.
```
Running State:
- Process is using CPU
- Instructions are being executed
- PC continuously updates
- Can transition to: Ready, Blocked, or Zombie
```

Q4: What happens when Context Switching is more frequent?
```
More Context Switching:
- Higher CPU overhead
- Lower system performance
- Decreased CPU efficiency
- Faster response time (benefit)
```

Q5: Explain the difference between Suspend and Terminate.
```
Suspend:
- Temporary
- Can be resumed
- State preserved

Terminate:
- Permanent
- Cannot be resumed
- State destroyed
```

Q6: What are the causes of Context Switching?
```
1. Time quantum expires
2. Higher priority process arrives
3. I/O request
4. Interrupt
5. System call
6. Process termination
```

Q7: Explain Suspended Ready state.
```
Suspended Ready:
- Process was in Ready state
- Suspended by OS
- Not ready to execute immediately
- Will move to Ready when resumed
```

Q8: What is the role of the Scheduler?
```
Scheduler:
- Selects process from Ready Queue
- Allocates CPU to selected process
- Types: Long-term, Short-term, Medium-term
- Determines which process runs next
```

Q9: Why does the OS suspend processes?
```
Reasons:
1. Memory Management (low RAM)
2. Power Saving (battery life)
3. Deadlock Recovery
4. Resource Management
```

Q10: What is the difference between Scheduler and Dispatcher?
```
Scheduler:
- Decides which process to run
- Makes selection decision
```

Dispatcher:
- Performs actual context switch
- Loads and saves contexts
```

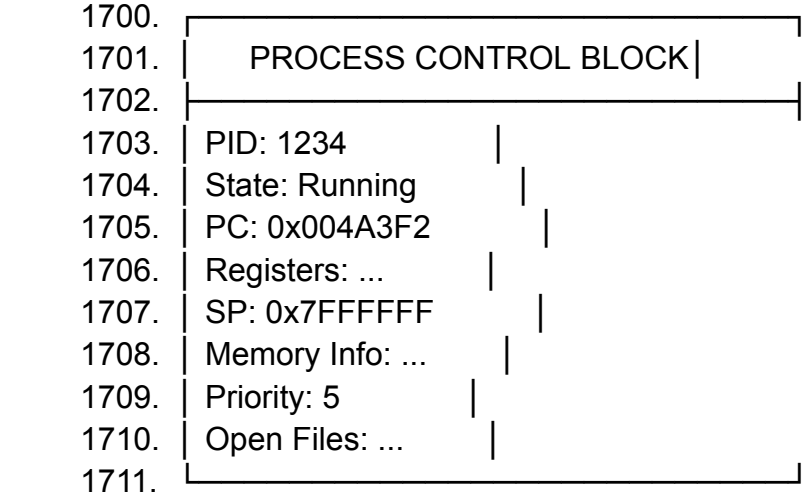
---

### 36. 5 Mark Questions with Answers

**Q1: Explain the complete Process Life Cycle with all states and transitions.**
```
PROCESS LIFE CYCLE:

1. NEW/EMBRYO State:
   - Process is being created
   - PID assigned
   - PCB created
   - Memory allocated
   - Transition: New → Ready (Admitted)

2. READY State:
   - Process is waiting for CPU
   - In Ready Queue
   - All resources allocated
   - Transition: Ready → Running (Scheduler dispatch)

3. RUNNING State:
   - Process using CPU
   - Instructions executing
   - Transition: Running → Ready (Time quantum expires)
   - Transition: Running → Blocked (I/O request)
   - Transition: Running → Zombie (Complete)

4. BLOCKED State:
   - Process waiting for I/O/Event
   - Not using CPU
   - Transition: Blocked → Ready (I/O complete)

5. ZOMBIE State:
   - Process completed
   - Only PCB remains
   - Waiting for parent to collect
   - Transition: Zombie → Destroyed

Complete Transitions:
New → Ready → Running → Ready (Time quantum)
Running → Blocked → Ready → Running
Running → Zombie → Destroyed
```

**Q2: Explain Context Switching in detail with its steps, advantages, and disadvantages.**
```
CONTEXT SWITCHING:

Definition:
The process of saving the context of one process and restoring the context of another.

Steps of Context Switching:

1. SAVE CURRENT CONTEXT:
   - Save Program Counter to PCB
   - Save CPU Registers to PCB
   - Save Stack Pointer to PCB
   - Save Process State to PCB

2. UPDATE PCB:
   - Change state: Running → Ready/Blocked
   - Update scheduling info

3. SCHEDULER RUNS:
   - Select next process from Ready Queue
   - Based on scheduling algorithm

4. LOAD NEW CONTEXT:
   - Load PC from new PCB
   - Load Registers from new PCB
   - Load Stack Pointer from new PCB

5. RESUME EXECUTION:
   - Start executing new process

ADVANTAGES:
- Enables multitasking
- Better CPU utilization
- Fair scheduling

- Improved responsiveness
- Supports time-sharing

DISADVANTAGES:
- CPU overhead
- Cache misses
- TLB flush (in many systems)
- Performance loss
- Increased latency
- Complex implementation
```

**Q3: Explain Preemptive and Non-Preemptive Scheduling with advantages, disadvantages, and examples.**
```
PREEMPTIVE SCHEDULING:

Definition:
OS can interrupt a running process and allocate CPU to another process.

Features:
- Process can be preempted
- Higher priority processes get CPU quickly
- Supports time-sharing

Advantages:
- Fast response time
- Better CPU utilization
- High priority processes execute quickly
- Supports interactive systems

Disadvantages:
- More context switching
- Higher CPU overhead
- Complex implementation

Examples:
- Round Robin (RR)
- Shortest Remaining Time First (SRTF)
- Preemptive Priority Scheduling

NON-PREEMPTIVE SCHEDULING:

Definition:

Process keeps CPU until it finishes or requests I/O.

Features:
- Process cannot be preempted
- CPU released voluntarily
- Simple implementation

Advantages:
- Less context switching
- Lower CPU overhead
- Simple implementation
- Less CPU time wasted

Disadvantages:
- High priority processes may wait
- Longer response time
- Convoy effect possible
- Process starvation possible

Examples:
- First Come First Served (FCFS)
- Shortest Job First (SJF)
- Non-Preemptive Priority Scheduling

COMPARISON:
Feature	Preemptive	Non-Preemptive
CPU Interruption	Yes	No
Context Switching	More	Less
Overhead	Higher	Lower
Response Time	Faster	Slower
Complexity	Higher	Lower
Best For	Interactive	Batch
```

**Q4: Explain Process Control Block (PCB) in detail, including its components and importance.**
```
PROCESS CONTROL BLOCK (PCB):

Definition:
PCB is a data structure maintained by the OS that contains all information about a process.

PCB COMPONENTS:

1. PROCESS ID (PID):
   - Unique identifier for the process

2. PROCESS STATE:
   - Current state: New, Ready, Running, Blocked, Zombie

3. PROGRAM COUNTER (PC):
   - Address of next instruction to execute

4. CPU REGISTERS:
   - General purpose registers content

5. STACK POINTER (SP):
   - Current stack top address

6. MEMORY MANAGEMENT INFO:
   - Page table base
   - Memory limits

7. SCHEDULING INFORMATION:
   - Process priority
   - Time quantum used

8. I/O STATUS INFORMATION:
   - List of open files
   - I/O operations in progress

9. ACCOUNTING INFORMATION:
   - CPU time used
   - Process start time

PCB LOCATION:
RAM (Main Memory)
└── Kernel Space
    └── Process Control Block (PCB)

IMPORTANCE OF PCB:

1. Process Identification:
   - Contains unique PID
   - Identifies process to OS

2. Context Switching:
   - Stores process context
   - Enables switching between processes

3. Process Management:
   - Tracks process state
   - Controls process execution

4. Resource Management:
   - Tracks allocated resources
   - Manages memory and I/O

5. Accounting:
   - Tracks CPU usage
   - Monitors process behavior

PCB STRUCTURE:




```

**Q5: Explain Process Suspension in detail, including its causes, operations, and comparison with termination.**

```
PROCESS SUSPENSION:

Definition:
Process Suspension is temporarily stopping a process without terminating it, while preserving its execution state.

SUSPENSION OPERATIONS:

A. STATE CHANGE:
- Process state changes to:

└── Suspended Ready (if was in Ready)
└── Suspended Blocked (if was in Blocked)

B. RESOURCE RELEASE:
- Removed from CPU execution
- CPU usage becomes 0%
- CPU time becomes available

C. CONTEXT SAVING:
- Save complete execution context:
└── CPU Registers
└── Program Counter (PC)
└── Stack Pointer (SP)
└── Stack
└── Other processor state

CAUSES OF SUSPENSION:

1. MEMORY MANAGEMENT:
- RAM is insufficient
- Less important processes suspended

2. POWER SAVING:
- Reduce power consumption
- Extend battery life

3. DEADLOCK RECOVERY:
- Resource conflicts
- System stability

4. USER REQUEST:
- Manual suspension

SUSPEND vs TERMINATE:

Feature	Suspend	Terminate
Duration	Temporary	Permanent
Resume	Yes	No
State	Preserved	Destroyed
CPU Usage	0%	Released
Resources	Held	Released
PCB	Retained	Destroyed

MEMORY MANAGEMENT:

RAM (Main Memory)
├── Kernel Space
│   └── PCB (Stays in RAM)
└── User Space
    └── Process Memory
        │
        │ (When low memory)
        ▼
    Secondary Storage
    └── Swap Space (Linux)
    └── Pagefile (Windows)

SWAP OUT:
- Process memory moved to disk
- PCB remains in RAM
- Process can be resumed later

SLEEP MODE:
- All processes suspended
- State preserved in RAM
- Computer enters low-power state
- Resume when woken

PROCESS STATE TRANSITION:
Running → Suspending → Suspended Ready
Running → Suspending → Suspended Blocked
Suspended Ready → Ready (Resume)
Suspended Blocked → Suspended Ready (I/O complete)
```

---

### 37. Viva Questions and Answers

**Q1: What is a process?**
> A process is a program in execution. It includes the program code, current activity, stack, data section, and process control block.

**Q2: What is the difference between program and process?**
> Program is a passive entity stored on disk, while process is an active entity loaded in memory and has execution state.
```

Q3: What is PCB and what does it contain?
> PCB (Process Control Block) is a data structure containing PID, process state, PC, registers, stack pointer, memory info, scheduling info, and I/O status.

Q4: Explain the various states of a process.
> New → Ready → Running → Blocked → Zombie. Process goes through these states during its lifetime.

Q5: What is context switching?
> Context switching is the process of saving the context of one process and loading the context of another.

Q6: What information is saved during context switching?
> Program Counter, CPU registers, Stack Pointer, and process state are saved.

Q7: Why is context switching needed?
> To enable multitasking, share CPU among processes, and provide fair scheduling.

Q8: What is the difference between preemptive and non-preemptive scheduling?
> Preemptive allows OS to interrupt processes, non-preemptive doesn't.

Q9: What is process suspension?
> Temporarily stopping a process without terminating it, preserving its state.

Q10: What is a zombie process?
> A process that has completed execution but its PCB remains until parent collects exit status.

Q11: What is the Ready Queue?
> A queue containing processes in Ready State waiting for CPU allocation.

Q12: What is swap space?
> Area on disk where OS moves process memory when RAM is insufficient.

Q13: What causes context switching?
> Time quantum expiry, higher priority process, I/O request, interrupt, system call, process termination.

Q14: What is dispatcher?
> Component that performs actual context switch and loads next process.

Q15: What is the difference between scheduler and dispatcher?
> Scheduler decides which process to run; dispatcher performs the actual switch.

Q16: What is Round Robin scheduling?
> Preemptive scheduling where each process gets a fixed time quantum in circular order.

Q17: What is priority scheduling?
> Scheduling based on process priority - higher priority processes get CPU first.

Q18: What is FCFS scheduling?
> Non-preemptive scheduling where processes are served in arrival order.

Q19: What is short-term scheduler?
> Selects process from Ready Queue for CPU allocation.

Q20: What is long-term scheduler?
> Selects processes to be loaded into memory.

Q21: What is medium-term scheduler?
> Removes/swaps processes from memory to disk.

Q22: What is a system call?
> Request from process to OS for service (I/O, file operations, etc.)

Q23: What is an interrupt?
> Signal from hardware or software requiring immediate attention.

Q24: What is I/O bound process?
> Process that spends more time on I/O operations than CPU.

Q25: What is CPU bound process?
> Process that spends more time on CPU operations than I/O.

---

38. MCQs for Practice

1. Which of the following is NOT a component of PCB?

a) PID
b) Program Counter
c) Process Memory (actual code)
d) Process State

Answer: c) Process Memory (actual code)

2. Context Switching is performed by:
a) Scheduler
b) Dispatcher
c) Both Scheduler and Dispatcher
d) Neither

Answer: c) Both Scheduler and Dispatcher

3. In which state does a process wait for I/O?
a) Ready
b) Running
c) Blocked
d) Zombie

Answer: c) Blocked

4. Preemptive Scheduling allows:
a) Process to run until completion
b) OS to interrupt running process
c) Process to request I/O
d) Process to terminate

Answer: b) OS to interrupt running process

5. Zombie state means:
a) Process is running
b) Process is waiting for I/O
c) Process completed but PCB remains
d) Process is suspended

Answer: c) Process completed but PCB remains

6. Process Suspension is:
a) Permanent termination
b) Temporary stopping
c) CPU allocation
d) Memory allocation

Answer: b) Temporary stopping

7. Which scheduler runs most frequently?
a) Long-term scheduler
b) Short-term scheduler
c) Medium-term scheduler
d) All run equally

Answer: b) Short-term scheduler

8. Context Switching overhead includes:
a) Saving registers
b) Loading registers
c) Cache effects
d) All of the above

Answer: d) All of the above

9. Which is an example of Preemptive Scheduling?
a) FCFS
b) SJF
c) Round Robin
d) Non-Preemptive Priority

Answer: c) Round Robin

10. PCB is stored in:
a) Secondary Storage
b) Kernel Space in RAM
c) User Space in RAM
d) Cache Memory

Answer: b) Kernel Space in RAM

11. Context switching happens when:
a) Time quantum expires
b) Higher priority process arrives
c) I/O request occurs
d) All of the above

Answer: d) All of the above

12. In Suspended Ready state:

a) Process is running
b) Process is waiting for CPU
c) Process is suspended and in memory/swap
d) Process has completed

Answer: c) Process is suspended and in memory/swap

13. Which is NOT an advantage of Preemptive Scheduling?
a) Fast response time
b) Better CPU utilization
c) Less context switching
d) High priority process execution

Answer: c) Less context switching

14. Which is an advantage of Non-Preemptive Scheduling?
a) Fast response time
b) Lower CPU overhead
c) Better CPU utilization
d) High priority process execution

Answer: b) Lower CPU overhead

15. PID stands for:
a) Process Information Data
b) Process Identification Data
c) Process ID
d) Program ID

Answer: c) Process ID

---

39. Important Definitions (Quick Revision)

Term	Definition
Process	A program in execution
PCB	Data structure containing all process information
PID	Unique identification number for process
Context Switching	Saving state of one process and loading state of another
Preemptive Scheduling	OS can interrupt and take CPU from process

Non-Preemptive Scheduling	Process keeps CPU until done
Process Suspension	Temporarily stopping a process
Zombie State	Process completed but PCB remains
Ready State	Process waiting for CPU
Running State	Process using CPU
Blocked State	Process waiting for I/O
Embryo State	Process being created
Scheduler	Selects process for CPU
Dispatcher	Performs context switch
Swap Space	Disk area for process memory

---

40. Important Diagrams for Exams

Diagram 1: Process State Diagram
```
New → Ready → Running → Zombie
Running → Ready (Time quantum)
Running → Blocked (I/O)
Blocked → Ready (I/O complete)
```

Diagram 2: Context Switching Flow
```
Process A Running
    ↓
Save Context to PCB A
    ↓
Select Next Process (Scheduler)
    ↓
Load Context from PCB B
    ↓
Process B Running
```

Diagram 3: PCB Structure
```
┌──────────────────────────┐
│      PCB         │
├──────────────────────────┤
│ PID              │
│ State            │
│ PC               │
```

```
2059. │ Registers        │
2060. │ SP               │
2061. │ Memory Info      │
2062. │ Scheduling Info  │
2063. │ I/O Status       │
2064. └──────────────────────────┘
2065. ```
2066.
2067. **Diagram 4: Memory Layout**
2068. ```
2069. Stack (High Address)
2070.     ↓
2071.   (Free Space)
2072.     ↑
2073. Heap
2074. Data Section
2075. Text Section
2076. ( Low Address)
2077. ```
2078.
2079. **Diagram 5: Scheduling Queues**
2080. ```
2081. Job Queue → Ready Queue → CPU
2082.             ↓
2083.         Device Queue
2084. ```
2085.
2086. **Diagram 6: Suspension Transition**
2087. ```
2088. Running → Suspending → Suspended Ready
2089. Running → Suspending → Suspended Blocked
2090. ```
2091.
2092. ---
2093.
2094. ### 41. Short Notes for Revision
2095.
2096. **Process:**
2097. > A process is a program in execution. It includes code, data, stack, heap, and PCB.
2098.
2099. **PCB:**
2100. > Data structure with all process info - PID, state, PC, registers, etc. Stored in kernel space.
```

States:
> New → Ready → Running → Blocked → Zombie

Context Switching:
> Saving one process context, loading another. Done by dispatcher.

Preemptive Scheduling:
> OS can interrupt and take CPU. More overhead, better response.

Non-Preemptive Scheduling:
> Process keeps CPU until done. Less overhead, simpler.

Suspension:
> Temporary stop. State preserved. Can be resumed.

Zombie:
> Complete but PCB remains. Parent collects exit status.

Scheduler:
> Selects process for CPU. Types: Long, Short, Medium-term.

Dispatcher:
> Performs actual context switch and context loading.

PID:
> Unique ID for each process. Assigned when created.

Swap Space:
> Disk area for moving process memory when RAM is low.

Sleep Mode:
> All processes suspended. State in RAM. Low power.

Context Switch Overhead:
> Time wasted on saving/loading contexts. Cache and TLB effects.

✅ Complete Chapter Checklist

- [x] Chapter 1: Introduction to Process
  - [x] Process
  - [x] Program vs Process

2145. - [x] PCB
2146. - [x] PID
2147.
2148. - [x] Chapter 2: Process States
2149. - [x] Embryo/New State
2150. - [x] Ready State
2151. - [x] Running State
2152. - [x] Blocked State
2153. - [x] Zombie State
2154. - [x] Complete State Diagram
2155.
2156. - [x] Chapter 3: Process Suspension
2157. - [x] Process Suspension
2158. - [x] Suspend vs Terminate
2159. - [x] Suspended Ready
2160. - [x] Suspended Blocked
2161. - [x] Swap Space
2162. - [x] Sleep Mode
2163.
2164. - [x] Chapter 4: Context Switching
2165. - [x] Context Switching Definition
2166. - [x] Information Saved
2167. - [x] Steps of Context Switching
2168. - [x] Overhead
2169. - [x] Advantages/Disadvantages
2170. - [x] More vs Less Context Switching
2171. - [x] Causes of Context Switching
2172.
2173. - [x] Chapter 5: Scheduling
2174. - [x] Preemptive Scheduling
2175. - [x] Non-Preemptive Scheduling
2176. - [x] Comparison
2177.
2178. - [x] Chapter 6: Scheduling Queues
2179. - [x] Job Queue
2180. - [x] Ready Queue
2181. - [x] Device Queue
2182.
2183. - [x] Chapter 7: Diagrams and Tables
2184. - [x] Complete Diagrams
2185. - [x] Comparison Tables
2186.
2187. - [x] Chapter 8: Exam Questions
2188. - [x] 1 Mark Questions

- [x] 2 Mark Questions
- [x] 5 Mark Questions
- [x] Viva Questions
- [x] MCQs
- [x] Definitions
- [x] Short Notes

** 📌 Important Points to Remember:**

1. **PCB** is in Kernel Space, not user space
2. **PID** is unique for each process
3. **Context Switching** saves only context, not program code
4. **Suspension** keeps PCB in RAM, may swap memory to disk
5. **Preemptive = OS can interrupt**, Non-Preemptive = Process keeps CPU
6. **Zombie = completed but not destroyed**
7. **Scheduler** decides, **Dispatcher** executes
8. **More context switching = More overhead, Better response**
9. **Less context switching = Less overhead, Worse response**
10. **Swap Space** is on disk, not in RAM

"Success is the sum of small efforts repeated day in and day out."

Good Luck for Your Exams! 📚🎯